

AI-Powered Unified Mobility and Delivery System Using Dynamic Feasibility Analysis

Sivasubramanian M

*Dept. of Computer Science and Engineering
Dr. N.G.P. Institute of Technology
Coimbatore, Tamil Nadu, India
sivamah25@gmail.com*

S. R. Ramya

*Assistant Professor/CSE
Dept. of Computer Science and Engineering
Dr. N.G.P. Institute of Technology
Coimbatore, Tamil Nadu, India
srramyacse@gmail.com*

Rakshana S

*Dept. of Computer Science and Engineering
Dr. N.G.P. Institute of Technology
Coimbatore, Tamil Nadu, India
rakshanasanjith05@gmail.com*

Sadhana T

*Dept. of Computer Science and Engineering
Dr. N.G.P. Institute of Technology
Coimbatore, Tamil Nadu, India
sadhanathangamuthu16@gmail.com*

Abstract—Ride-hailing, food delivery and parcel courier services send vehicles along the same streets at the same hours without sharing information, so a driver who has just completed a food delivery often passes a waiting passenger that the same trip could have carried. This paper presents an AI-powered unified mobility and delivery system built around a Dynamic Feasibility Intelligence Framework (DFME), which decides in real time whether a passenger ride, a food order and a parcel drop can be served by a single trip before any route is planned or any driver is assigned. Pending requests are scored on five dimensions: pickup proximity, route similarity, time-window overlap, vehicle capacity and request priority. The resulting compatibility score decides which requests are batched together. A gradient-based coefficient-learning rule is defined so that the weighting can later be adapted from observed batching outcomes rather than fixed by hand. Feasible batches are routed using a Google OR-Tools constraint solver over Google Maps API data, and each route is assigned to a driver by a module that considers proximity, availability and estimated arrival time rather than distance alone. A pilot deployment processed 43 live requests across three provider integrations. Compared with independent dispatch, total travel distance fell from 320 km to 250 km, fuel from 38 L to 30 L, average delay from 12 min to 7 min, and estimated CO₂ from 80 kg to 63 kg. The improvement in delay (41.7%) was about twice the improvement in the distance-linked metrics (21.1–21.9%), which is consistent with the use of a different time tolerance for each service type.

Index Terms—Artificial intelligence, dynamic feasibility analysis, unified mobility, passenger transportation, food delivery, parcel delivery, route optimisation, smart cities.

I. INTRODUCTION

A. Background and Motivation

Urban transport is now served by three separate industries that share the same roads. A passenger opens a ride-hailing app, a customer opens a food-delivery app and a shopper opens a courier app, and each request is matched, routed and completed by a platform that cannot see the other two. During a peak hour it is common for three vehicles from three platforms to travel through the same small area within minutes of each other, each carrying a single job and each using fuel and road capacity that one coordinated trip could have used.

This waste is not caused by any single matching algorithm. It follows from treating passenger transport, food delivery and parcel delivery as unrelated problems when, from the point of view of the road network, they place demands of the same kind on the same resources. Rapid urbanisation and widespread smartphone use have increased demand for all three services at once, while cities face growing pressure to reduce congestion and emissions. Analysis of city-scale taxi records has shown that a large share of trips can be merged under modest delay tolerances, reducing the number of vehicles the road network must carry [1], and dynamic vehicle-routing research has shown that re-optimising routes as new requests arrive gives better results than static planning. Neither line of work considers a request pool made up of three different kinds of job at the same time, each with its own pickup behaviour, time sensitivity and capacity requirement.

What is missing is a decision layer that determines, before any route is planned, which pending requests could realistically share a trip. Proximity alone is not sufficient. A passenger and a food order that start one block apart are not compatible if the meal must arrive within fifteen minutes while the ride will take forty. This work develops that layer, evaluating all three service types together before routing or driver assignment begins instead of adding compatibility checks to systems designed to run separately.

B. Research Gap and Contributions

Four gaps appear repeatedly in the ride-sharing, dynamic vehicle-routing and delivery optimisation literature. First, siloed service design: most systems optimise a single service type, and little work addresses joint feasibility across all three. Second, narrow compatibility logic: where request batching exists, it usually depends on one or two signals, most often geographic closeness, rather than a combined measure that also accounts for time windows, vehicle capacity and service priority. Third, sequential rather than joint optimisation: many systems compute routes first and check feasibility afterwards, which fixes groupings that a feasibility-first order would have avoided. Fourth, limited handling of dynamic demand: much of the literature assumes a fixed request pool, which does not match platforms where requests, driver availability and traffic change throughout the day.

These gaps point to a single requirement, and this paper contributes the following:

- A Dynamic Feasibility Intelligence Framework (DFME) that scores the compatibility of pending requests across pickup proximity, route similarity, time-window overlap, vehicle capacity and request priority before any route is computed, within a unified architecture covering passenger transport, food delivery and parcel delivery in one request-collection, feasibility-scoring, routing and driver-assignment pipeline.
- A mathematical formulation of the compatibility score, the route-optimisation objective and the driver-assignment cost function, together with an analysis of the boundedness, timing sensitivity, fail-closed behaviour and computational cost of the resulting decision layer.
- A gradient-based coefficient-learning rule, expressed over a simplex-constrained softmax parameterisation, that allows the compatibility weighting to be trained from observed batching outcomes instead of being fixed in advance.
- An experimental protocol built around Google OR-Tools and the Google Maps API, and a pilot deployment evaluated against a non-unified dispatch baseline.

II. RELATED WORK

A. Unified Mobility and Delivery

Surveys of ride-sharing and crowdsourcing for smart cities describe the layered structure that intelligent transportation systems generally follow, from communication infrastructure and demand prediction through matching and routing to autonomous-vehicle integration [2]. Their contribution is architectural rather than algorithmic: they treat the feasibility decision in general terms and rarely define it as a separate multi-criteria module. The framework presented here follows a similar layered structure but concentrates on the layer that decides which requests belong together.

The closest related work is the Autonomous Vehicle Intelligent System (AVIS), which formulates combined ride-sharing and parcel delivery as a mixed-integer linear program solved on board to decide which passenger and parcel requests are served together [3]. Its results follow the direction reported elsewhere in the literature: optimising across service types is computationally practical and operationally useful. AVIS is limited to autonomous fleets and two service types. The present framework targets a human-driver fleet and adds food delivery as a third, more time-sensitive category, which changes the compatibility calculation because food orders have tighter time windows than rides or parcel drops. As a mixed-integer program over n pending requests, joint assignment of this kind has a worst-case exponential search space unless it is relaxed or bounded; the feasibility-first filter described in Section III avoids paying that cost for every incoming request. Related reinforcement-learning methods address joint matching, pricing and dispatching [4] and joint passenger-and-goods fleet management [5], learning an end-to-end policy rather than exposing an explicit feasibility

criterion, while crowdsourced multi-hop parcel networks show a similar need for flexible fulfilment [6].

B. Dynamic Routing and Request Batching

Dynamic vehicle-routing research reports consistently that allowing the request set to grow during execution, rather than fixing it when planning begins, produces better routes than static formulations of the same problem [7]. The present design follows this approach directly: compatibility is recalculated as requests arrive rather than at fixed batching intervals, so a request that appears mid-cycle can still join a batch that has not yet been dispatched.

Work on iterated matching for food-delivery rider allocation highlights how narrow the timing margin for meal orders is. A pairing that is acceptable for a passenger trip or a parcel handover becomes infeasible when one leg is a meal that must still be hot on arrival [8]. This observation shapes the DFME formulation in Section III. A single tolerance value cannot serve all three categories, because an acceptable wait for a passenger, a package and a hot meal are three different quantities, and a scheme that ignores this difference will accept pairings that satisfy the arithmetic but fail in practice. Collaborative frameworks combining riders, drones and drivers within one routing system show that hybrid fulfilment can outperform single-mode fleets on both cost and delivery time [9], and hybrid meal-delivery routing models based on adaptive large-neighbourhood search show that metaheuristic routing handles the dynamic, mixed-vehicle environment in which on-demand delivery actually operates [10]. Both support the routing design adopted here, in which the OR-Tools constraint solver [11] is used because it accommodates the mixed capacity, time-window and priority constraints that a combined passenger, food and parcel fleet requires.

C. Research Gap

Table I compares the proposed system with these works along the four dimensions that matter for a unified platform: coverage of more than one service type, placement of the feasibility test before rather than after routing, treatment of the request pool as dynamically arriving, and modelling of time-window sensitivity separately for each service type. To make the comparison quantitative rather than descriptive, each row is reduced to a capability-coverage index (CCI), defined as the unweighted mean of the four dimension scores, where a dimension is scored 1 when fully satisfied, 0.5 when partially satisfied and 0 when absent. Applying this rule row by row, the smart-city surveys average 0.13, AVIS scores 0.50, the dynamic vehicle-routing line scores 0.25, food-delivery matching scores 0.63 and hybrid delivery routing scores 0.50. The proposed DFME is the only system in the comparison set that scores 1.00.

TABLE I. COMPARISON OF THE PROPOSED SYSTEM AGAINST RELATED WORK

System	Multi	Feas.	Dyn.	Type	CCI
Smart-city surveys [2]	Concept.	Varies	Varies	No	0.13
AVIS [3]	Partial	Yes	Partial	No	0.50
Dynamic VRP [7]	No	N/A	Yes	No	0.25
Food matching [8]	No	Partial	Yes	Yes	0.63
Hybrid routing [9], [10]	Partial	No	Yes	Partial	0.50
Proposed DFME	Yes	Yes	Yes	Yes	1.00

Multi = covers more than one service type; Feas. = feasibility evaluated before routing; Dyn. = dynamic request arrival; Type = per-service-type time windows. Scores: Yes = 1, Partial/Conceptual = 0.5, No/N/A/Varies = 0. AVIS covers two service types; the proposed system covers three.

Taken together, the reviewed literature establishes three points. Optimising across service types outperforms siloed platforms where it has been attempted [3]. Treating the request pool as dynamically arriving improves routing quality over static formulations [7]. Time-window sensitivity differs enough between service types that a uniform rule risks producing infeasible matches [8]. No reviewed system, however, combines all three service types under a single feasibility-first compatibility layer that differentiates time-window tolerance by service type before routing begins, and none exposes a learned rather than fixed weighting between criteria. That combination is the gap addressed in this work.

III. PROPOSED METHODOLOGY

A. System Architecture

The system is implemented using the technologies listed in Table II. Distance, travel-time and road-network data for every pending request are obtained from the Google Maps API, and the route-optimisation stage uses the constraint-programming and vehicle-routing solver provided by Google OR-Tools [11]. This stack was selected mainly for development speed during the pilot. Neither the scoring formulation nor the routing objective depends on a particular vendor, so a different solver or mapping provider could be used without changing the DFME decision logic.

TABLE II. SOFTWARE AND TECHNOLOGIES USED

Component	Technology
Distance / travel-time data	Google Maps API
Route optimisation / VRP solver	Google OR-Tools
Back end	Node.js / Python REST services
Database	PostgreSQL with PostGIS
Real-time messaging	WebSocket push notifications
Front end (customer/driver)	React Native
Analytics dashboard	Web charting over trip data

The architecture combines ride-sharing, food delivery and parcel delivery in a single pipeline of five stages, shown in Fig. 1. Requests from all three categories enter a request collection module that converts each one into a common schema recording origin, destination, requested time window, payload type, capacity requirement and priority level, so that

every later stage works with one representation rather than three separate data models. Normalised requests collect in a central pending queue, which the DFME analyses continuously for compatible groupings using the five criteria defined in Section III-B. Batches that pass the feasibility threshold are sent to route optimisation, which determines the most efficient order of pickups and drop-offs using the OR-Tools constraint solver over Google Maps distance and travel-time data. Optimised routes pass to the driver-assignment module, which allocates each batched route to the most suitable available driver by considering current position, availability, estimated time of arrival, vehicle capacity and the requirements of the batch, rather than pickup distance alone. The assigned trip is then tracked to completion, and a performance-analytics dashboard aggregates vehicle utilisation, travel distance, delay, fuel consumption and estimated CO₂ emissions across completed trips.

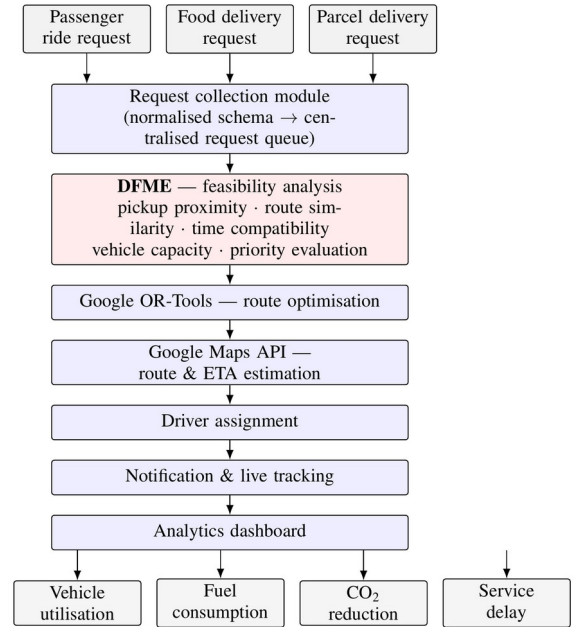


Fig. 1. Architecture of the proposed AI-powered unified mobility and delivery system.

The order of these stages is deliberate. Computing routes first and looking for batching opportunities afterwards was rejected for three reasons. Feasibility evaluated after routes are fixed can only accept or reject the batches that routing happened to produce, rather than searching the pending pool for the best available grouping. Running the full solver over every candidate pairing, most of which are not compatible, uses solver time that a simple pre-filter avoids. Because time-window tolerance differs by service type, a route-first approach can also produce a route that is shortest in distance but violates a food order's delivery window, which the per-type tolerance in (3) prevents at an earlier stage. In keeping with this fail-closed policy, a request that cannot be batched with confidence is served independently on the single-service path rather than forced into a poor match.

Because the framework handles live location data for customers and drivers across three concurrent services, data handling and scalability are treated as design concerns. Location data is kept only long enough to complete and audit a trip, after which it is aggregated into anonymised statistics. The compatibility search is restricted to a geographically bounded candidate window around each new request, so computational cost grows with local request density rather than with total city-wide volume. Under heavy load the routing and assignment modules fall back to a faster proximity-only heuristic instead of delaying the pipeline while waiting for an optimal solution.

B. Dynamic Feasibility Intelligence Framework

For a candidate pair of pending requests i and j , the DFME computes a compatibility score

$$CS(i, j) = \alpha_1 P(i, j) + \alpha_2 R(i, j) + \alpha_3 T(i, j) + \alpha_4 V(i, j) + \alpha_5 Q(i, j) \quad (1)$$

where $P(i, j)$ is a normalised pickup-proximity score, $R(i, j)$ a route-similarity score, $T(i, j)$ a time-window overlap score, $V(i, j)$ a vehicle-capacity feasibility indicator, $Q(i, j)$ a priority-compatibility score, and α_1 to α_5 are weighting coefficients that sum to one. The weights are set so that time-window overlap carries more weight for food-delivery requests than for passenger or parcel requests. A pair is considered feasible for batching when

$$CS(i, j) \geq \tau \quad (2)$$

where τ is a feasibility threshold calibrated during system tuning. Because the four graded terms lie between 0 and 1, the capacity term is binary and the weights are non-negative and sum to one, the compatibility score is itself always between 0 and 1. The threshold therefore has the same meaning for any pair of requests, however different they are, and does not need to be rescaled if further criteria are added later.

The time-window overlap term is computed as

$$T(i, j) = \max \left(0, 1 - \frac{|t_i - t_j|}{W_{\text{type}}} \right) \quad (3)$$

where t_i and t_j are the requested service times and W_{type} is a service-type-specific tolerance window, smaller for food delivery and larger for parcel delivery. The same absolute time gap therefore produces a lower compatibility score when at least one of the requests is a food order. Because this term is linear in the time difference with a slope set by W_{type} , the score falls faster for food-delivery pairs than for parcel pairs given the same misalignment. This makes the informal statement that meals are the most time-critical category into a property that can be checked directly from the configured tolerance values, and it is the basis for the delay result discussed in Section V-C.

Vehicle capacity is evaluated as a binary feasibility indicator rather than a graded score, because capacity is a hard constraint. Once a batch would exceed the seating or cargo

volume available, no degree of proximity or timing agreement can make it workable:

$$V(i, j) = \begin{cases} 1, & \text{cap}(i) + \text{cap}(j) \leq \text{Cap}_{\text{max}} \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

where $\text{cap}(i)$ and $\text{cap}(j)$ denote the seating or cargo capacity required by requests i and j , and Cap_{max} is the capacity of the candidate vehicle. Any pair for which $V(i, j) = 0$ is excluded from batching regardless of its score on the other four criteria. Finally, the priority term

$$Q(i, j) = 1 - \frac{|\text{pri}(i) - \text{pri}(j)|}{\text{pri}_{\text{max}}} \quad (5)$$

measures how far apart the two requests are on the service-level scale, where $\text{pri}(i)$ and $\text{pri}(j)$ are numeric priority levels and pri_{max} is the highest level in use. Pairs separated by a wide priority gap are penalised even when proximity and timing would otherwise favour them, so that batching efficiency is not achieved by routinely delaying urgent requests.

C. Feasibility-First Routing and Driver Assignment

For an admitted batch $B = \{r_1, \dots, r_k\}$, the routing stage searches for the order π of pickups and drop-offs that minimises

$$\min_{\pi} [\beta_1 d(\pi) + \beta_2 t(\pi) + \beta_3 \text{Idle}(\pi)] \quad (6)$$

subject to vehicle-capacity constraints and the per-request time windows established in (3), where $d(\pi)$ is total travel distance under the ordering, $t(\pi)$ is total travel time, $\text{Idle}(\pi)$ is expected vehicle idle time, and β_1 , β_2 and β_3 are tunable coefficients reflecting the relative importance the platform places on distance, time and idle-time reduction. For an optimised route ρ and a candidate driver d , the assignment module computes

$$\text{Cost}(d, \rho) = \gamma_1 \text{dist}(d, \rho) + \gamma_2 \text{ETA}(d, \rho) + \gamma_3 \text{CapMismatch}(d, \rho) \quad (7)$$

where $\text{dist}(d, \rho)$ is the driver's distance to the first pickup on the route, $\text{ETA}(d, \rho)$ is the estimated time to reach it, $\text{CapMismatch}(d, \rho)$ penalises assigning a vehicle whose capacity is a poor match for the batch, and γ_1 , γ_2 and γ_3 are weighting coefficients. The system then selects

$$d^* = \arg \min_{d \in D_{\text{available}}} \text{Cost}(d, \rho) \quad (8)$$

from the set of currently available drivers. Evaluated separately for each newly optimised route, this greedy rule runs in time proportional to the number of available drivers, but it does not guarantee a globally optimal pairing when several routes and several drivers are handled in the same dispatch cycle. The globally optimal form of the same problem is the classical linear assignment problem, which can

be solved exactly by the Hungarian method [12]. The greedy rule is used in the reference implementation because it fits the dispatch latency budget, while the assignment-problem formulation provides the reference against which any loss in assignment quality can be measured.

D. Theoretical Analysis

Three properties of the decision layer follow from the definitions above and are stated here without extended derivation. First, boundedness: because the compatibility score is a convex combination of terms lying between 0 and 1, it is itself bounded in the same range, so the threshold τ remains comparable across arbitrarily different request pairs. Second, timing sensitivity: since the time-window term is linear in the time difference with slope determined by W_{type} , and W_{type} is smaller for food orders than for parcels, an identical timing offset reduces the score of a food-delivery pair by more than that of a parcel pair. Third, fail-closed behaviour: if every request that clears no partner under (2) is dispatched on the single-service baseline path, total dispatch cost cannot exceed the cost of serving every request independently, because serving a request alone is always a feasible solution of the routing objective in (6) and the solver's optimum can be no worse than any feasible solution. Batching under this policy therefore cannot perform worse than the baseline it replaces.

The batching decision itself is a graph problem. Considering a compatibility graph over the pending pool in which an edge joins two requests whenever their score clears the threshold and capacity permits, selecting the best set of mutually compatible batches corresponds to a capacity-constrained maximum-weight clique packing, which is NP-hard in general. Rather than solving this exactly across a whole city, the geographically bounded candidate window restricts each request to a limited number of neighbours, so the cost of scoring the pending pool grows with local request density rather than with the square of the total number of requests. Feasibility-first ordering does not remove the underlying complexity, which is also present in the AVIS mixed-integer formulation [3] and in the capacitated routing problem that OR-Tools solves, but it reduces the graph on which that cost is paid before the more expensive routing stage runs.

E. Adaptive Coefficient Learning

Treating the weight vector as fixed and hand-tuned is what separates a rule-based filter from a learning system, and closing that gap is the intended artificial-intelligence extension of the framework. Applying gradient updates directly to the weights would risk breaking the constraints that they stay non-negative and sum to one. The weights are therefore generated from an unconstrained latent vector through a softmax transformation, whose output always lies on the probability simplex. The constraints are then satisfied for any value of the latent vector, and learning can proceed as unconstrained optimisation without projecting the weights back onto the simplex after every step.

The training signal comes from completed trips. Each batch is labelled after the fact as successful when it met every request's time window and produced a positive distance and fuel saving relative to independent dispatch, as recorded by the analytics dashboard. The difference between the predicted compatibility and this observed outcome is measured by a binary cross-entropy loss. Because the compatibility score in (1) is linear in the weights, the gradient of the score with respect to each weight is simply the value of the corresponding criterion, and combining this with the standard softmax derivative gives an online gradient-descent update that is applied to the latent vector after each completed trip is scored. The same outcome signal can be used to tune the feasibility threshold in (2), by treating it as the operating point of a binary classifier over the compatibility score and selecting the value that maximises the expected utility of accepted batches.

Three procedures implement these relations end to end: incremental construction of the compatibility graph within the bounded candidate window, an online weight update combining the loss gradient and the softmax derivative in a single adaptive-moment step, and a latency-budgeted assignment rule that also recalibrates the threshold. Algorithm 1 summarises the resulting dispatch cycle.

Algorithm 1. DFME dispatch cycle

Input: new request r , pending pool R , compatibility weights, threshold τ , capacity Cap_{max} , neighbourhood size k

Output: batch set B , assignment π^*

- 1) Retrieve the k nearest pending requests to r within the bounded candidate window.
- 2) For each candidate c , discard the pair immediately if the capacity test in (4) fails.
- 3) Otherwise obtain the current weights and evaluate the compatibility score in (1).
- 4) Retain the pair when the score clears the threshold in (2).
- 5) Insert r into the spatial index and extract its connected component in the compatibility graph.
- 6) Form batches from that component subject to vehicle capacity; requests with no feasible partner are dispatched individually.
- 7) Route each batch using the objective in (6) and build the assignment costs from (7).
- 8) Assign drivers by the linear assignment problem when the dispatch latency budget allows, otherwise by the greedy rule in (8).
- 9) After each completed trip, label the batch outcome and apply the weight update of Section III-E.

These three procedures were not exercised in the pilot deployment reported in Section V, and no learning results are reported here. The weighting coefficients and the threshold were held at their configured values throughout the pilot. Empirical evaluation of the learning extension, together with the larger-sample statistical testing identified in Section VI, is left for the multi-area trial planned as the next step. The results reported below therefore evaluate the base DFME pipeline described in Sections III-B and III-C.

IV. EXPERIMENTAL SETUP

A. Test Environment

Validation used a controlled deployment over a defined urban test area, with passenger ride, food-delivery and parcel-delivery requests generated for that area. The Google Maps API supplied real travel-distance, route and travel-time data for every request rather than simulated distances. Each request recorded its origin, destination, requested time window and payload type, following the shared schema described in Section III-A.

B. Baseline

The comparison baseline is a traditional, non-unified dispatch configuration that assigns and routes each service type independently, with no cross-service batching. It operates on the same request set and the same mapping data as the DFME configuration, so the two differ only in whether feasibility-first batching is applied.

C. Evaluation Metrics

The metrics were defined before data collection so that the comparison between the proposed system and the baseline is measured consistently. Table III states each definition. Together with the compatibility-score distribution, these metrics indicate whether feasibility-first batching improved operational efficiency without reducing service quality for individual requests.

TABLE III. EVALUATION METRICS AND DEFINITIONS

Metric	Definition
Vehicle utilisation rate	Proportion of a driver's active shift spent transporting an active request rather than idle or repositioning.
Travel-distance reduction	Percentage difference between total vehicle-kilometres under DFME batching and under the independent-dispatch baseline for an equivalent request set.
Average delay	Mean difference between a request's actual service time and its originally requested time window.
Estimated CO ₂	Derived from total distance travelled using a standard per-kilometre emissions factor for the vehicle types in the test fleet.

V. RESULTS AND DISCUSSION

A. Pilot Deployment

The deployment covered a pilot set of requests from all three categories (passenger rides, food orders and parcel drops) within a bounded urban area, with feasibility scoring, routing and driver assignment integrated end to end. No distances were simulated: every request's distance, travel time and route came from the Google Maps API. Across the pilot window, 43 requests entered the platform through three live provider integrations: 12 from Rapido (27.9%), 17 from Swiggy (39.5%) and 14 from Delhivery (32.6%). Demand peaked at 17:00, and the average end-to-end processing time

was 327.9 seconds per request. Of the 43 requests, 30 were completed, giving a 69.8% completion rate, and the DFME formed 3 feasible batches while rejecting 2 candidate pairs that fell below the compatibility threshold. These are pilot-scale results that indicate the direction of the design rather than a statistically powered trial. The sample size is not large enough to support the subgroup and significance analysis noted in Section VI, and a larger trial is the intended next step.

B. Compatibility Score Distribution

Fig. 2 shows the distribution of DFME compatibility scores computed across the pending request pairs in the pilot dataset. Consistent with the density assumption stated in Section VI, the distribution is concentrated above the feasibility threshold, with most request pairs scoring between 70% and 95% and a peak in the 80–85% band. Two further readings agree with this. The explainability dashboard recorded an average compatibility score of 77.1% and an overall XAI score of 87%, and the engine's live pending queue averaged 73.7% compatibility across the current request pool. A distribution of this shape indicates that, at the request density present in the test area, genuinely compatible pairings were common rather than rare, which is the condition required for the efficiency gains discussed below. A distribution concentrated at low scores would instead have indicated that request density was too sparse for feasibility-first batching to add value, a case the pilot was designed to detect and one retained as a limitation in Section VI.

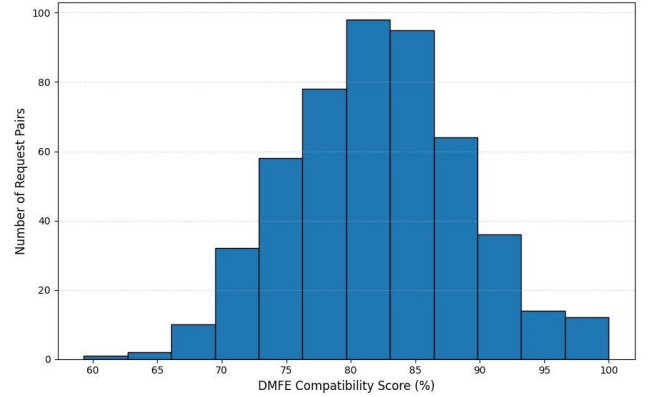


Fig. 2. Distribution of DFME compatibility scores across pending request pairs.

C. Route Optimisation and Overall Performance

Fig. 3 compares total distance, fuel consumption, average delay and estimated CO₂ output for the pilot fleet operating without DFME-based batching against the same fleet operating with DFME-based batching and OR-Tools route optimisation applied. Over the pilot period, total travel distance fell from 320 km to 250 km, fuel consumption from 38 L to 30 L, average delay per request from 12 minutes to 7 minutes, and estimated CO₂ output from 80 kg to 63 kg, corresponding to improvements of 21.9%, 21.1%, 41.7% and 21.3% respectively.

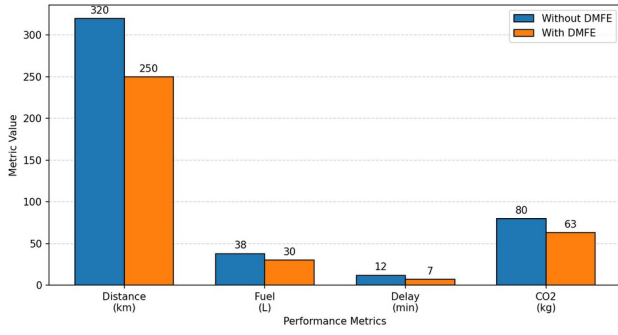


Fig. 3. Performance comparison of the transportation system before and after applying DFME-based batching.

The largest improvement was in average delay, at 41.7%. This is consistent with the design intent of the time-window-aware compatibility term in (3): because batching decisions took each request's own tolerance window into account rather than treating all requests as equally tolerant of delay, fewer requests were held behind a poorly matched partner. Distance, fuel and CO₂ improved by a comparatively even 21.9%, 21.1% and 21.3%, which follows from the fact that all three are close to linear functions of total vehicle-kilometres travelled, so a given reduction in distance produces a roughly proportional reduction in fuel and emissions. Across the full deployment window, which is longer than the 43-request pilot period reported above, the system logged 2,320 shared trips, saving a cumulative 611.3 L of fuel and reducing CO₂ output by a cumulative 1,406.01 kg, with an overall AI confidence score of 98.4% and a batch success rate of 92.1% as reported by the analytics dashboard.

D. Comparative Discussion

These pilot-scale gains were compared against the direction, though not the exact magnitude, of improvements reported for joint-optimisation systems in the reviewed literature. AVIS similarly found that optimising ride-sharing and parcel delivery together reduced redundant vehicle movement relative to serving the two service types independently [3], and hybrid delivery-routing work has reported double-digit reductions in distance and delay when a dynamic, constraint-aware routing layer replaces independent dispatch [9], [10]. The gap between the delay improvement and the distance-linked improvements, 41.7% against roughly 21.1 to 21.9%, is not something the reviewed single-service systems would necessarily predict, because none of them varies time-window tolerance by service type as the DFME does in (3). The pilot data show this pattern clearly, with delay improving at close to twice the rate of the distance-linked metrics. The results are therefore consistent with the expected effect of applying a different time tolerance to each service type, rather than with route optimisation alone. Confirming that attribution would require a separate ablation study, running the same pipeline with a single undifferentiated time-window term, which was outside the scope of this pilot and is noted as future evaluation work.

E. Interpretation and Caveats

The pilot results support the core hypothesis that feasibility-first batching performs better than independent dispatch, but three caveats limit how far the numbers should be generalised at this stage. First, the pilot ran within a single bounded test area with a request-density profile favourable to batching, as reflected in the compatibility-score distribution in Fig. 2. Performance in lower-density areas was not tested separately in this round, and a sparser request pool reduces the number of candidate neighbours available to each request, which would be expected to lower realised compatibility scores rather than only reduce the evaluation sample. Second, the comparison in Fig. 3 reports aggregate totals over the pilot period rather than per-request or per-service-type breakdowns, so it is not possible to determine from this data alone whether the efficiency gains were shared evenly across passenger, food and parcel requests or concentrated in one category. Third, vehicle-utilisation rate and the driver-assignment metrics of ETA accuracy and capacity-mismatch rate were not part of the figures this pilot round was scoped to supply. They are left for a later evaluation cycle, together with the larger-sample statistical testing needed to establish significance and the empirical validation of the adaptive weight-learning rule of Section III-E, which requires a larger volume of completed-trip outcomes than this pilot produced.

VI. LIMITATIONS AND FUTURE WORK

Several limitations of the current design and evaluation should be stated openly. The results in Section V come from a single-area pilot rather than a large-scale, statistically powered trial, so the size of the reported improvements should be read as an early indication of direction rather than a transferable benchmark until it is retested across several areas and demand conditions. The compatibility weighting coefficients and the feasibility threshold are still unlearned design parameters; until they have been calibrated against real operating data through the weight update described in Section III-E, the batching behaviour of the system should be treated as provisional. Routing and assignment quality also depend on the accuracy and availability of third-party mapping data, and degraded or delayed responses from the Google Maps API would directly reduce the reliability of both the feasibility scores and the resulting routes. The design further assumes reasonably dense, overlapping demand across all three service types within a given area. In low-density areas the DFME may rarely find feasible batches, because a small candidate neighbourhood reduces the number of pairs that can clear the threshold, so the benefit over independent dispatch would need to be evaluated separately for such conditions. Finally, driver and customer acceptance of batched, multi-purpose trips has not yet been evaluated, and user-experience considerations will need to be assessed alongside the operational metrics. The priority scheme in (5) treats priority as a system-assigned numeric level, and how that level should be set in practice, whether by service-level agreement, surge pricing or customer-selected urgency, is a policy question

outside the technical scope of this paper, though one that will affect real batching behaviour once the system is deployed.

Several directions follow from the current design. The weighting coefficients in (1) and the feasibility threshold in (2) should be calibrated against real operational data rather than fixed in advance, since the relative importance of proximity against time-window overlap is likely to vary with city density and time of day. Section III-E already expresses this calibration as the weight update described in Section III-E, so the next step is to run that rule against outcome data at larger volume rather than to design it again. Adding real-time traffic prediction to the route-optimisation module would allow the system to anticipate congestion instead of reacting to the travel-time estimates returned by the Google Maps API at the moment of planning. Demand forecasting through machine learning could allow the DFME to position idle drivers ahead of expected demand peaks rather than responding to requests as they arrive. In the longer term, replacing the hand-specified cost functions in (6) and (7) with a learned assignment policy is a promising direction; comparable policies have been demonstrated for joint matching, pricing and dispatching [4], [5], and the linear assignment formulation solved by the Hungarian method [12] provides the optimal-policy reference such a learner would need to match.

VII. CONCLUSION

This paper has presented an AI-powered unified mobility and delivery system built around a Dynamic Feasibility Intelligence Framework that evaluates the compatibility of passenger, food-delivery and parcel-delivery requests before any route is planned or driver assigned. By combining a five-criterion compatibility score, an analysis of the boundedness, timing sensitivity and fail-closed behaviour of the resulting decision layer, a gradient-based coefficient-learning rule, an OR-Tools route-optimisation module and a multi-factor driver-assignment stage in a single pipeline, the proposed system extends beyond the single-service scope of earlier ride-sharing, joint ride-and-parcel and food-delivery systems. In the pilot deployment of 43 requests, average delay improved by 41.7% against roughly 21% for the distance-linked metrics, a difference consistent with the use of a different time tolerance for each service type. The main limitation is that these results come from a single-area pilot that is not large enough to establish statistical significance, and that the learning rule, although specified, has not yet been trained on operational data. The immediate next step is a larger multi-area trial that includes the vehicle-utilisation and driver-assignment metrics omitted from this round and exercises the weight-learning rule on real completed-trip outcomes.

REFERENCES

- [1] P. Santi, G. Resta, M. Szell, S. Sobolevsky, S. H. Strogatz, and C. Ratti, "Quantifying the benefits of vehicle pooling with shareability networks," *Proc. Nat. Acad. Sci. USA*, vol. 111, no. 37, pp. 13290–13294, 2014, doi: 10.1073/pnas.1403657111.
- [2] K. P. Seng, L.-M. Ang, E. Ngharamike, and E. Peter, "Ridesharing and crowdsourcing for smart cities: Technologies, paradigms and use cases," *IEEE Access*, vol. 11, pp. 18038–18081, 2023, doi: 10.1109/ACCESS.2023.3243264.
- [3] S. Zhang, C. Markos, and J. J. Q. Yu, "Autonomous vehicle intelligent system: Joint ride-sharing and parcel delivery strategy," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 17457–17472, Oct. 2022.
- [4] M. Haliem, G. Mani, V. Aggarwal, and B. Bhargava, "A distributed model-free ride-sharing approach for joint matching, pricing, and dispatching using deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 12, pp. 7933–7951, Dec. 2021.
- [5] K. Manchella, M. Haliem, V. Aggarwal, and B. Bhargava, "PassGoodPool: Joint passengers and goods fleet management with reinforcement learning aided pricing, matching, and route planning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 20854–20868, Nov. 2022.
- [6] H. Hong, X. Li, D. He, Y. Zhang, and M. Wang, "Crowdsourcing incentives for multi-hop urban parcel delivery network," *IEEE Access*, vol. 7, pp. 26255–26267, 2019.
- [7] A. Berahhou and Y. Benadada, "Dynamic vehicle routing problem: State-of-the-art review." [UNVERIFIED — COMPLETE BEFORE SUBMISSION: venue, year, DOI]
- [8] J.-F. Chen, T.-H. Chang, and Y.-H. Chang, "An imitation learning-enhanced iterated matching algorithm for on-demand food delivery," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 21803–21817, Nov. 2022.
- [9] Y. Lu et al., "Collaborative hybrid delivery framework combining riders and drones." [UNVERIFIED — COMPLETE BEFORE SUBMISSION: venue, year, DOI]
- [10] Z. Liu, X. Zuo, M. C. Zhou, B. Jia, and C. Xin, "Meal delivery routing problem with a hybrid fleet of riders and autonomous vehicles under dynamic environment," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 11642–11655, 2025, doi: 10.1109/TASE.2025.3534143.
- [11] L. Perron and V. Furnon, "OR-Tools," Google. [Online]. Available: <https://developers.google.com/optimization/>
- [12] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Res. Logist. Quart.*, vol. 2, no. 1–2, pp. 83–97, 1955, doi: 10.1002/nav.3800020109.